

Appeler des fonctions

Objectifs d'apprentissage

- Comprendre ce que sont les fonctions et pourquoi elles sont géniales.
- Comprendre comment fonctionnent les fonctions.
- Comprendre comment lire la documentation des fonctions.

Vous avez déjà travaillé avec de nombreuses fonctions `R` ; les commandes telles que `getwd()` , `length()` et `unique()` sont toutes des fonctions. Vous savez qu'une commande est une fonction car elle est suivie de parenthèses `()` .

Tout comme les **variables** sont des noms pratiques utilisés pour appeler des *objets* tels que des vecteurs ou des tableaux de données, les **fonctions** sont des noms pratiques pour appeler des *processus* ou des *actions*. Une fonction `R` est simplement un ensemble de code qui effectue une certaine action.

Les variables représentent des données, tandis que les fonctions représentent du code.

La plupart des fonctions comportent trois éléments clés : (1) une ou plusieurs entrées, (2) un processus appliqué à ces entrées et (3) une sortie du résultat. Lorsque vous appelez une fonction dans `R` , vous dites : " Hé `R` , prends ces informations, fais quelque chose avec et renvoie-moi le résultat. " Vous fournissez les entrées à la fonction, et celle-ci s'occupe du reste.

Prenons l'exemple de la fonction `mean()` . `mean()` calcule la moyenne arithmétique (c'est-à-dire la moyenne) d'un ensemble de valeurs.

```
x <- c(4,6,3,2,6,8,5,3) # créer un vecteur de nombres
mean(x) # trouver leur moyenne
## [1] 4.625
```

Dans cette commande, vous fournissez à la fonction `mean()` l'entrée `x` .

Cette analogie vous aidera peut-être à comprendre : lorsque vous pensez aux fonctions, pensez aux distributeurs automatiques : vous donnez deux entrées au distributeur automatique (votre argent et votre choix de snack), puis il vérifie si votre choix est en stock et si l'argent que vous avez fourni est suffisant pour payer le snack que vous voulez. Si c'est le cas, la machine renvoie une sortie (le snack).

Fonctions de base dans `R`

Il existe des centaines de fonctions déjà intégrées à `R` . Ces fonctions sont appelées "*fonctions de base*". Tout au long de ces modules, nous vous avons présenté, et continuerons de vous présenter, les fonctions de base les plus couramment utilisées.

Vous pouvez accéder à d'autres fonctions grâce à des ensembles de codes externes appelés *paquets*, que nous expliquerons dans un prochain module.

Vous pouvez également écrire vos *propres* fonctions (et vous le ferez !). Nous proposons un module entier sur la manière de procéder.

Notez que toutes les fonctions ne nécessitent pas d'entrée. La fonction `getwd()` , par exemple, n'a besoin

d'aucune entrée entre parenthèses pour trouver et renvoyer votre répertoire de travail actuel.

Enregistrement du résultat d'une fonction

Il est presque toujours conseillé d'enregistrer le résultat d'une fonction dans une nouvelle variable. Sinon, la fonction se contente d'afficher son résultat dans la *console* et R l'oublie.

Vous pouvez enregistrer le résultat d'une fonction de la même manière que n'importe quelle autre valeur :

```
x <- c(4,6,3,2,6,8,5,3)
x_mean <- mean(x)
x_mean
## [1] 4.625
```

Fonctions à entrées multiples

Notez que `mean()` accepte un second argument appelé `na.rm`. Il s'agit de l'abréviation de `NA.remove`. Lorsque cet argument est défini sur `TRUE`, R supprime les valeurs manquantes ou invalides du vecteur avant de calculer la moyenne.

```
x <- c(4,6,3,2,NA,8,5,3) # notez la valeur NA
mean(x,na.rm=TRUE)
## [1] 4.428571
```

Si vous tentiez d'exécuter ces commandes avec `na.rm` défini sur `FALSE`, R générerait une erreur et s'arrêterait.

Notez que vous avez fourni à la fonction `mean()` deux arguments, `x` et `na.rm`, séparés par une virgule. C'est ainsi que l'on passe plusieurs arguments à une fonction.

Valeurs par défaut des fonctions

Notez que de nombreuses fonctions ont des valeurs par défaut pour leurs arguments. Si vous ne spécifiez pas vous-même la valeur d'un argument, R utilisera la valeur par défaut. Dans le cas de `mean()`, la valeur par défaut de `na.rm` est `FALSE`. Cela signifie que le code suivant générera une erreur :

```
x <- c(4,6,3,2,NA,8,5,3) # notez les valeurs manquantes
mean(x)
## [1] NA
```

Comme R suppose que vous utilisez la valeur par défaut de `na.rm`, qui est `FALSE`, cela signifie que vous ne souhaitez pas supprimer les valeurs manquantes avant de calculer la moyenne.

Documentation des fonctions (obtenir de l'aide)

Les fonctions sont conçues pour accepter un nombre limité d'arguments, chacun avec un nom spécifique. Pour connaître les entrées attendues d'une fonction et les résultats attendus, consultez sa page d'aide :

```
## starting httpd help server ... done
```

Lorsque vous saisissez cette commande, la documentation d'aide de `mean()` s'affiche dans le panneau inférieur droit de votre fenêtre RStudio :



Apprendre à lire cette documentation est essentiel pour maîtriser R .

Attention : la documentation n'est pas toujours facile à comprendre ! Vous finirez par détester une documentation mal rédigée et apprécierez d'autant plus une documentation bien faite ; les quelques minutes supplémentaires que l'auteur de la fonction consacre à la rédaction d'une bonne documentation épargnent aux utilisateurs du monde entier des heures de frustration et de confusion.

- Les sections `Title` et `Description` vous aident à comprendre le rôle de cette fonction.
- La section `Usage` vous montre comment saisir le code de la fonction.
- La section `Arguments` liste chaque argument possible (qui, dans le jargon de R , est un synonyme d'*entrée* ou de *paramètre*), explique ce que chaque entrée est attendue et détaille les exigences de formatage.
- La section `Value` décrit la valeur renvoyée par la fonction.
- En bas de la page d'aide, un exemple de code est fourni pour illustrer le fonctionnement de la fonction. Vous pouvez copier-coller ce code dans votre propre script ou dans la console et observer les résultats.

Notez que les fonctions plus complexes peuvent également inclure une section “ Détails ” dans leur documentation, qui fournit davantage d'explications sur le fonctionnement de la fonction, les types d'entrées requis et la valeur de retour.

Exemples de fonctions

R propose un ensemble de fonctions de base pour les **statistiques descriptives**, qui illustrent bien le fonctionnement des fonctions et leur utilité.

On peut utiliser le même vecteur comme entrée pour toutes ces fonctions :

```
x <- c(4,6,3,2,NA,8,9,5,6,1,9,2,6,3,0,3,2,5,3,3) # notez la valeur NA
```

La fonction `mean()` a été expliquée ci-dessus.

```
result <- mean(x,na.rm=TRUE)
result
## [1] 4.210526
```

`median()` renvoie la médiane du vecteur fourni :

```
result <- median(x,na.rm=TRUE)
result
## [1] 3
```

sd() renvoie l'écart type du vecteur fourni :

```
result <- sd(x,na.rm=TRUE)
result
## [1] 2.594416
```

summary() renvoie un vecteur décrivant plusieurs aspects de la distribution du vecteur :

```
result <- summary(x,na.rm=TRUE)
result
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NA's
##    0.000   2.500   3.000   4.211   6.000   9.000        1
```

Exercices

Enquête sur les chaussettes

Vous avez mené une enquête auprès de vos camarades de classe, en leur demandant combien de paires de chaussettes ils possèdent. La plupart ont répondu, mais une personne a refusé de vous répondre. Au lieu de vous donner un nombre, elle a simplement dit : “ Non ! ”.

Vos résultats ressemblent à ceci :

```
# Personnes interrogées
peers <- c('Keri','Eric','Joe','Ben','Matthew','Jim')

# Leurs réponses
socks <- c(6,8,14,1,NA,4)
```

1. Calculez le nombre moyen de paires de chaussettes possédées par vos camarades.
2. Calculez le nombre total de paires de chaussettes possédées par les élèves coopératifs de cette classe.
3. Utilisez le code pour trouver le nom de la personne qui a refusé de vous parler de ses chaussettes.
4. Utilisez le code pour trouver les noms des personnes qui ont accepté de participer à votre enquête.
5. Utilisez le code pour trouver le nom de la personne qui possède le plus de chaussettes.
6. Utilisez le code pour trouver le nom de la personne qui possède le moins de chaussettes.

Enquête sur l'âge

7. Créez un vecteur nommé `years` contenant les années de naissance de toutes les personnes présentes.
8. Quelle est l'année de naissance moyenne ?
9. Quelle est l'année de naissance médiane ?
10. Créez un vecteur appelé `ages` contenant l'âge (approximatif) de chaque personne.
11. Quel est l'âge minimum ?

12. Quel est l'âge maximum ?
13. Quel est l'âge médian ?
14. Quel est l'âge moyen ?
15. Résumez le vecteur `ages` .
16. Quelle est l'étendue des âges ?
17. Quel est l'écart type des âges ?
18. Consultez l'aide de la fonction `sort()` .
19. Créez un vecteur appelé `sorted_ages` . Il faudrait trier les âges.
20. Consultez la fonction `length()` .
21. Combien de personnes composent le groupe ?
22. Créez un objet appelé `old` . Attribuez à cet objet un âge (par exemple 36) à partir duquel une personne est considérée comme " âgée ".
23. Créez un objet appelé `old_people` . Il doit s'agir d'un vecteur booléen indiquant si chaque personne est âgée ou non.
24. La septième personne dans `ages` est-elle âgée ?
25. À combien d'années la personne 12 est-elle considérée comme vieille ou jeune ?

Lancer de dés

26. Consultez la page d'aide de la fonction `sample()` .

Voici un exemple d'utilisation de cette fonction. Cette ligne de code sélectionne un nombre aléatoire parmi les nombres compris entre 1 et 10.

```
sample(1:10,size=1)
## [1] 2
```

Cette commande génère trois nombres aléatoires :

```
sample(1:10,size=3)
## [1] 1 7 4
```

27. Utilisez cette fonction pour simuler le lancer d'un dé.
28. Utilisez-la maintenant pour simuler 10 lancers de dé. (*Remarque* : consultez l'argument `replace` de `sample()` .)
29. Lancez le dé 10 000 fois et assignez le résultat à une nouvelle variable. (*Remarque* : consultez l'argument `replace` de `sample()` .)
30. Consultez la page d'aide de la fonction `table()` .
31. Utilisez la fonction `table()` pour déterminer si le dé obtenu à la question 29 est équilibré, ou s'il est biaisé. Pouvez-vous décrire le fonctionnement de la fonction `table()` ?

30. Utilisez maintenant la fonction `sample()` pour résoudre un autre problème : vos amis veulent commander à emporter au bar, mais personne parmi vous quatre ne veut aller chercher la commande. Écrivez un code qui désignera aléatoirement la personne qui devra aller la chercher.